Comparaison d'Algorithmes de Tri

Travail dirigé n°4

Algorithmique et Programmation

Imène Ben El Khaznadji & **Fatima Saadna**

Licence 3 CMI — Université Paris Nanterre

Date de rendu : 28 novembre 2025

Chargé de TD : Valentin Bouquet

Table des matières

Introduction **3**

1 Environnement de test **3**

2 Algorithmes de tri **4**

2.1 Tri à Bulle 4
2.2 Tri Cocktail 4
2.3 Tri par Insertion 4
2.4 Tri Selection 5
2.5 Tri Fusion 5
2.6 Tri Rapide 5
2.7 Tri par Dénombrement 6
2.8 Tri par Tas 6
2.9 Tri Gnome 6
2.10 Tri par tas (bonus) 7
2.11 Tri gnome (bonus) 7

3 Complexités théoriques **7**

4 Protocole expérimental **7**

4.1 Méthodologie 8
4.2 Mesure du temps 8

5 Résultats expérimentaux **8**

5.1 Graphiques comparatifs 8
5.1.1 Comparaison globale sur données aléatoires 9
5.1.2 Comparaison des algorithmes efficaces 10
5.1.3 Impact des optimisations : Tri à bulle (meilleur cas) 11
5.1.4 Pire cas : Impact du tableau décroissant 12
5.1.5 Comparaison Tri à Bulle vs Tri Fusion (meilleur cas) 13
5.1.6 Comparaison Tri à Bulle vs Tri Fusion (pire cas) 14
5.1.7 Comparaison Tri Fusion vs Tri Rapide (cas aléatoire) 15
5.2 Récapitulatif des performances 16
5.3 Analyse des performances 16
5.3.1 Correspondance avec les complexités théoriques 16
5.3.2 Comportements surprenants observés 17
5.3.3 Algorithme le plus performant en pratique 17

6 Conclusion **17**

Introduction

Ce travail dirigé s'inscrit dans le cadre du module **Algorithmique et Programmation**. Il a pour objectif de comparer expérimentalement les performances de plusieurs **algorithmes de tri classiques**, en mesurant leurs temps d'exécution selon la taille et la nature des données (aléatoires, triées, inversées).

Les tris étudiés sont :

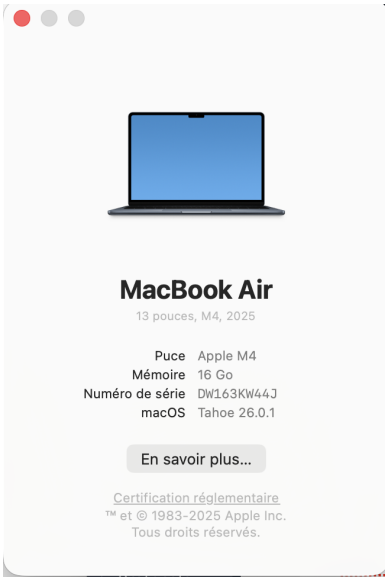
- Tri à bulle (3 versions)
- Tri cocktail
- Tri par sélection
- Tri par insertion
- Tri par dénombrement
- Tri fusion
- Tri rapide
- Tri par tas (bonus)
- Tri gnome (bonus)

1 Environnement de test

Les expérimentations ont été réalisées sur deux machines Apple Silicon récentes :

MacBook Air M2 (2022)

Processeur : Apple M2
Mémoire : 16 Go
Système : macOS Sequoia 15.4.1
Langage : C
Compilateur : GCC 13.2.0



MacBook Air M4 (2025)

Processeur : Apple M4
Mémoire : 16 Go
Système : macOS Tahoe 26.0.1
Langage : C
Compilateur : GCC 14.1.0

Les deux configurations offrent une excellente stabilité et permettent d'observer des performances cohérentes entre les générations M2 et M4.

2 Algorithmes de tri

2.1 Tri à Bulle

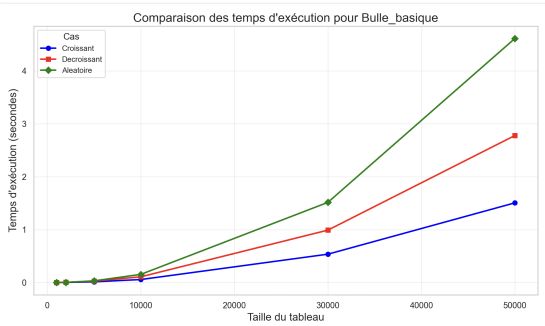


FIGURE 1 – Comparaison des temps d'exécution pour le Tri à Bulle.

2.2 Tri Cocktail

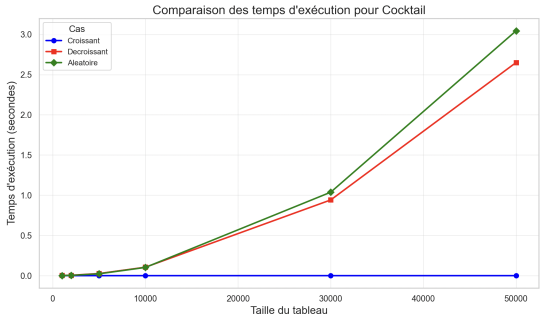


FIGURE 2 – Comparaison des temps d'exécution pour le Tri Cocktail.

2.3 Tri par Insertion

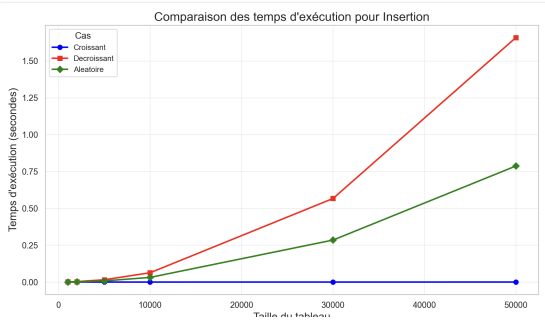


FIGURE 3 – Comparaison des temps d'exécution pour le Tri par Insertion.

2.4 Tri Selection

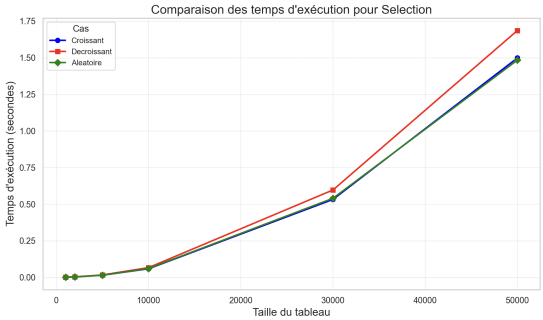


FIGURE 4 – Comparaison des temps d'exécution pour le Tri Selection.

2.5 Tri Fusion

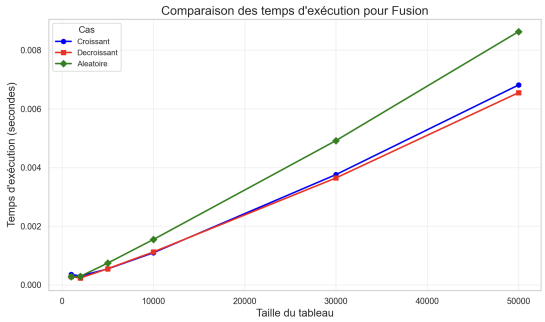


FIGURE 5 – Comparaison des temps d'exécution pour le Tri Fusion.

2.6 Tri Rapide

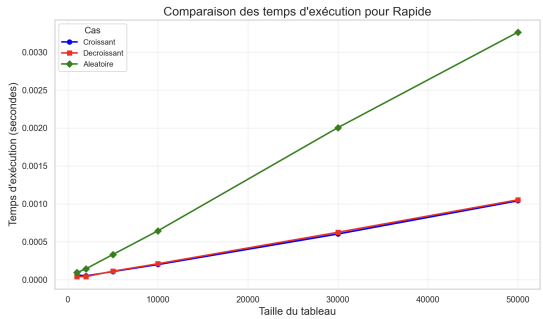


FIGURE 6 – Comparaison des temps d'exécution pour le Tri Rapide.

2.7 Tri par Dénombrement

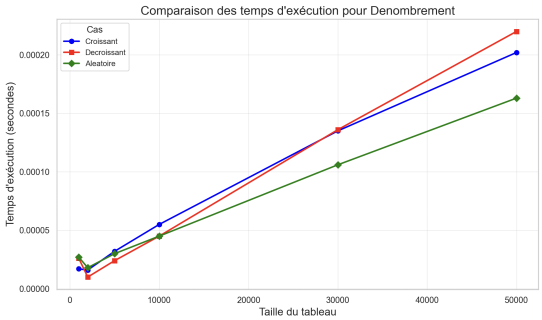


FIGURE 7 – Comparaison des temps d'exécution pour le Tri par Dénombrement.

2.8 Tri par Tas

FIGURE 8 – Comparaison des temps d'exécution pour le Tri par Tas.

2.9 Tri Gnome

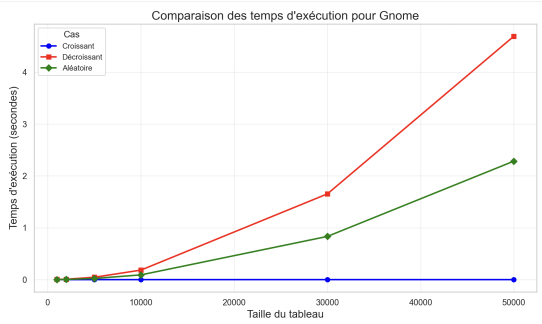


FIGURE 9 – Comparaison des temps d'exécution pour le Tri Gnom.

En bonus, nous avons également implémenté et testé deux algorithmes supplémentaires :

2.10 Tri par tas (bonus)

Le tri par tas utilise une structure de tas (arbre binaire) pour organiser les éléments, puis extrait successivement le maximum pour construire le tableau trié. Complexité : $O(n \log n)$ garantie. Il trie sur place sans nécessiter de mémoire supplémentaire significative, ce qui le rend particulièrement intéressant lorsque la mémoire est une contrainte.

2.11 Tri gnome (bonus)

Le tri gnom est une variante du tri par insertion qui déplace un élément en arrière tant qu'il est mal placé, comme un gnome de jardin qui range des pots. Complexité : $O(n^2)$. Bien que peu utilisé en pratique en raison de ses performances limitées sur de grandes données, il reste intéressant d'un point de vue pédagogique pour comprendre les mécanismes de tri par comparaison.

3 Complexités théoriques

Algorithme	**Meilleur cas**	**Cas moyen**	**Pire cas**
Tri à bulle	$O(n)$	$O(n^2)$	$O(n^2)$
Tri cocktail	$O(n)$	$O(n^2)$	$O(n^2)$
Tri par sélection	$O(n^2)$	$O(n^2)$	$O(n^2)$
Tri par insertion	$O(n)$	$O(n^2)$	$O(n^2)$
Tri par dénombrement	$O(n+k)$	$O(n+k)$	$O(n+k)$
Tri fusion	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$
Tri rapide	$O(n \log n)$	$O(n \log n)$	$O(n^2)$
Tri par tas	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$
Tri gnom	$O(n)$	$O(n^2)$	$O(n^2)$

Table 1 – Complexités temporelles des algorithmes de tri.

Note : Les algorithmes avec une complexité en $O(n^2)$ sont adaptés aux petites données (moins de 1000 éléments), tandis que ceux en $O(n \log n)$ sont recommandés pour de grands volumes.

4 Protocole expérimental

4.1 Méthodologie

Les tests ont été effectués sur des tableaux de tailles ($n = 1000, 2000, 5000, 10000, 30000, 50000$), avec différents cas d'usage représentatifs :

- **Tableau trié** : éléments en ordre croissant (meilleur cas pour certains algorithmes)
- **Tableau inversé** : éléments en ordre décroissant (pire cas pour plusieurs algorithmes)
- **Tableau constant** : tous les éléments ont la même valeur
- **Tableau aléatoire** : valeurs générées aléatoirement (cas moyen)

Note : Notez que pour chaque algorithme, l'échelle du graphique a été ajustée en fonction des variations de temps d'exécution observées.

4.2 Mesure du temps

Le temps d'exécution a été mesuré à l'aide de la fonction `clock()` de la bibliothèque standard `<time.h>`. Chaque algorithme a été exécuté plusieurs fois afin d'obtenir une moyenne représentative, en isolant uniquement le temps de tri (sans génération des tableaux).

Les graphiques ont été générés avec **Python** (bibliothèque Matplotlib) pour les comparaisons globales, et avec **RStudio** pour l'analyse du tri à bulle sur données décroissantes.

5 Résultats expérimentaux

5.1 Graphiques comparatifs

5.1.1 Comparaison globale sur données aléatoires

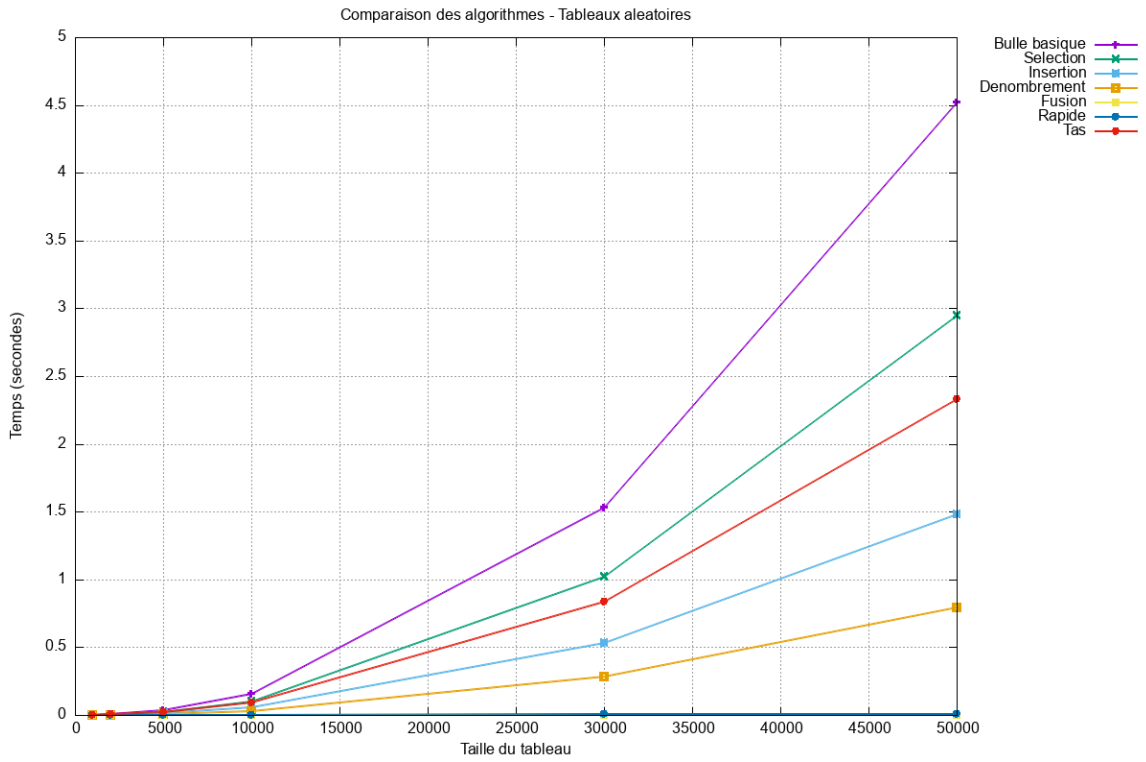


FIGURE 10 – Comparaison des temps d'exécution de tous les algorithmes testés (données aléatoires) — Généré avec Python.

Ce graphique permet d'observer clairement la différence de performance entre les algorithmes quadratiques (Bulle, Sélection, Insertion) et les algorithmes en $O(n \log n)$ (Fusion, Rapide, Tas). On constate que :

- Le tri à bulle est le plus lent, avec une croissance quadratique marquée
- Les tris par sélection et par insertion ont des performances similaires
- Le tri par dénombrement reste linéaire et très performant
- Les tris fusion et rapide sont les plus efficaces pour les grandes tailles

5.1.2 Comparaison des algorithmes efficaces

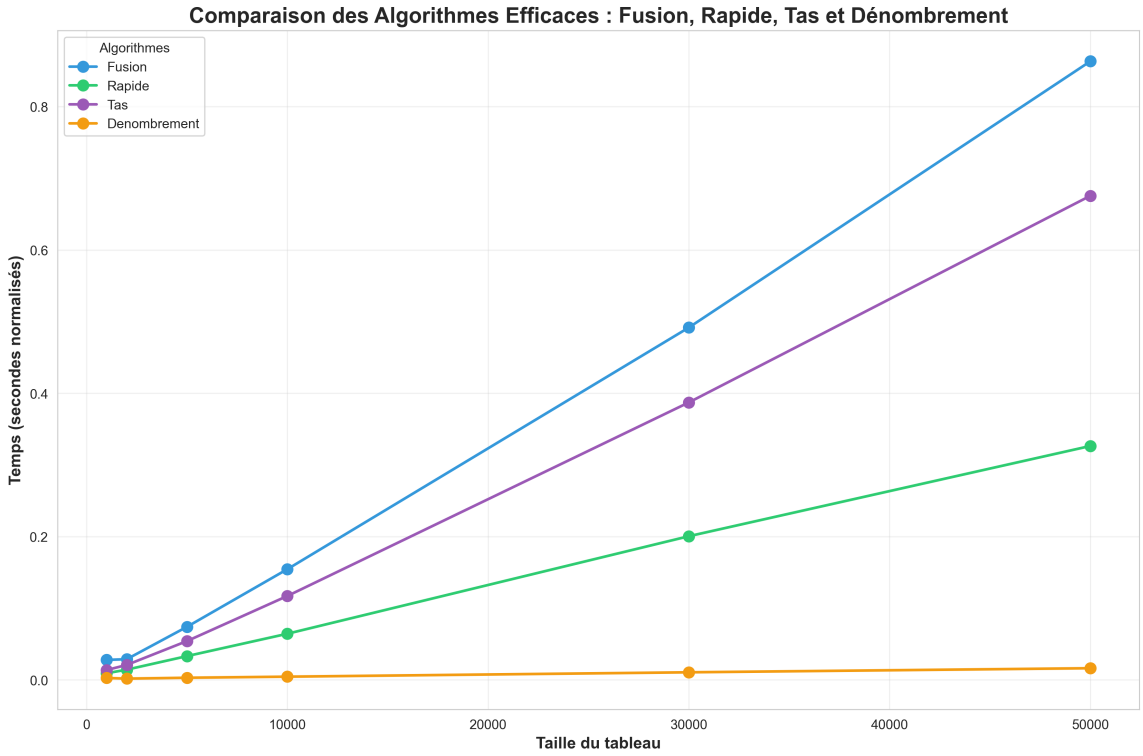


FIGURE 11 – Zoom sur les algorithmes performants : Fusion, Rapide, Tas et Dénombrement — Généré avec Python.

Ce graphique met en évidence les performances des algorithmes les plus efficaces en isolant leurs courbes. On observe que :

- Le tri par dénombrement reste quasi-constant (linéaire) même pour 50000 éléments
- Le tri rapide est le plus performant des algorithmes en $O(n \log n)$
- Le tri par tas suit une croissance similaire au tri fusion
- La différence entre fusion et rapide s'accentue avec l'augmentation de la taille

5.1.3 Impact des optimisations : Tri à bulle (meilleur cas)

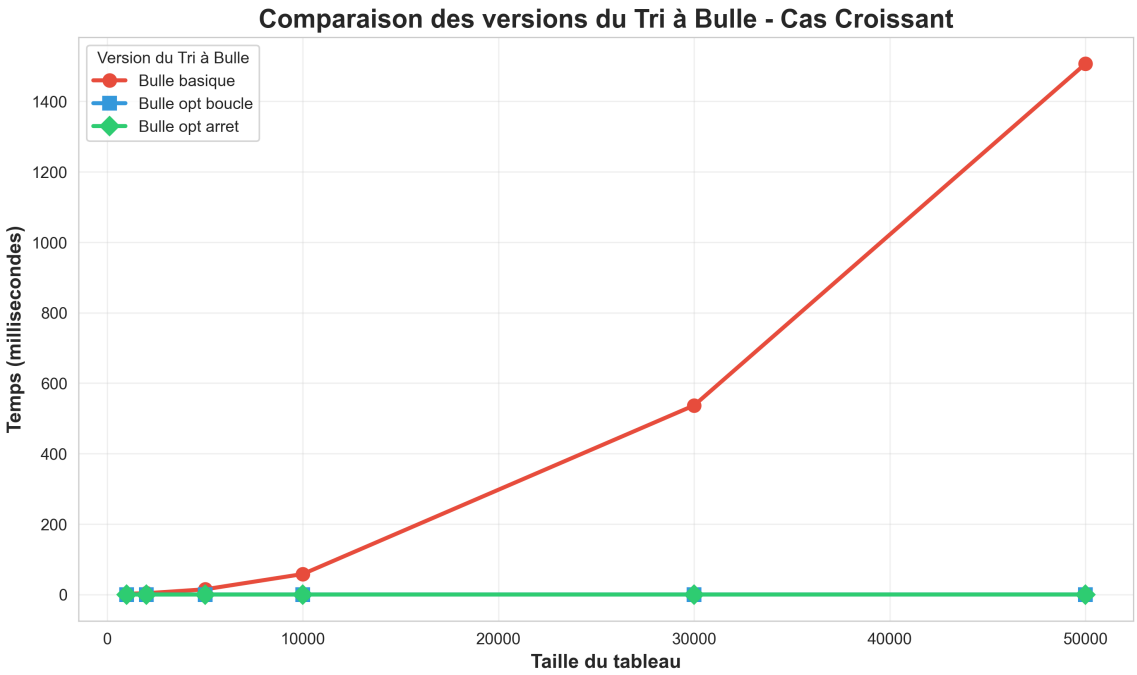


FIGURE 12 – Comparaison des trois versions du tri à bulle sur données croissantes (meilleur cas) — Généré avec Python.

Ce graphique illustre l'impact des optimisations sur le tri à bulle dans le meilleur cas (données déjà triées) :

- **Bulle basique** : aucune optimisation, complexité $O(n^2)$ même si déjà trié
- **Bulle opt boucle** : avec flag de détection, arrêt anticipé si déjà trié $\rightarrow O(n)$
- **Bulle opt arrêt** : réduction de zone + flag $\rightarrow$ performances optimales
- Les versions optimisées sont quasi-instantanées sur données triées

5.1.4 Pire cas : Impact du tableau décroissant

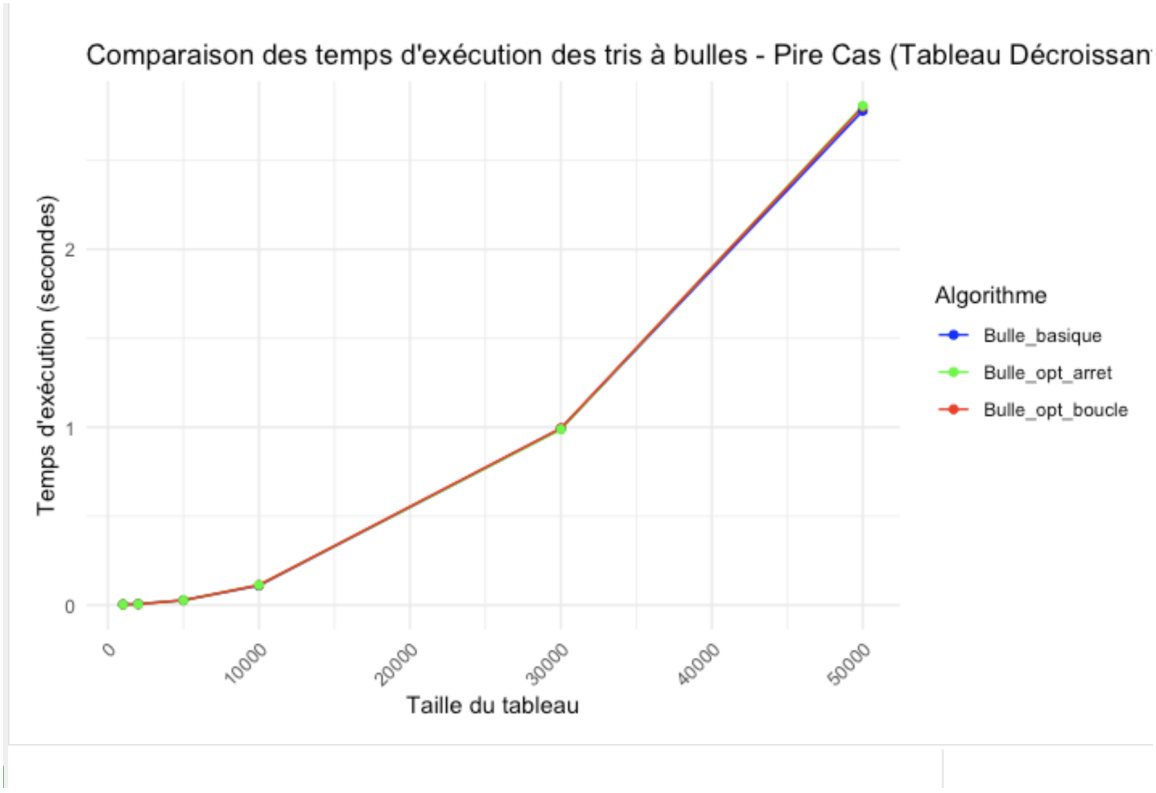


FIGURE 13 – Comparaison des trois versions du tri à bulle sur données décroissantes (pire cas) — Généré avec RStudio.

Dans le pire cas (données inversées), les trois versions du tri à bulle montrent des performances quasi-identiques, confirmant que les optimisations n'aident que lorsque le tableau est partiellement ou totalement trié.

5.1.5 Comparaison Tri à Bulle vs Tri Fusion (meilleur cas)

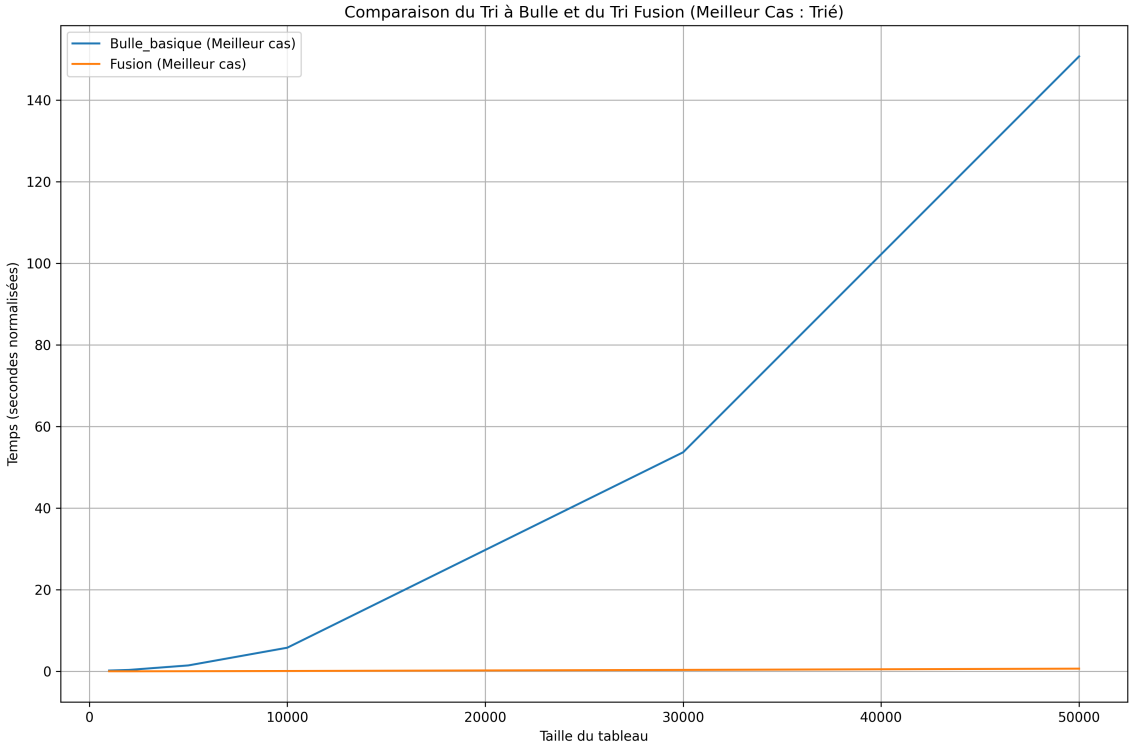


FIGURE 14 – Comparaison du Tri à Bulle et du Tri Fusion dans le meilleur cas (tableau trié) — Généré avec Python.

Le tri à bulle montre une performance significativement plus lente que le tri fusion, bien que l'optimisation du tri à bulle permette d'améliorer légèrement la situation dans ce cas particulier. Le tri fusion reste bien plus performant avec une complexité en $O(n \log n)$ contre $O(n^2)$ pour le tri à bulle.

5.1.6 Comparaison Tri à Bulle vs Tri Fusion (pire cas)

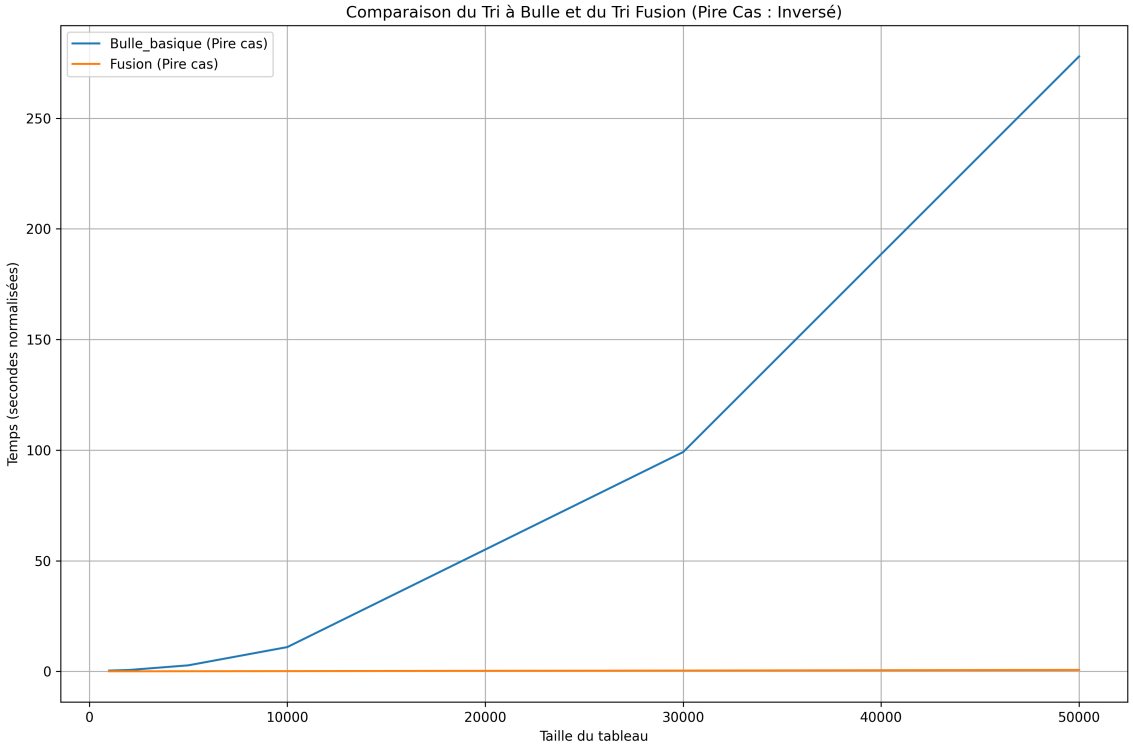


FIGURE 15 – Comparaison du Tri à Bulle et du Tri Fusion dans le pire cas (tableau inversé) — Généré avec Python.

Dans le pire cas, le tri à bulle atteint sa complexité maximale $O(n^2)$ sur un tableau inversé, tandis que le tri fusion, avec sa complexité $O(n \log n)$, reste bien plus performant.

5.1.7 Comparaison Tri Fusion vs Tri Rapide (cas aléatoire)

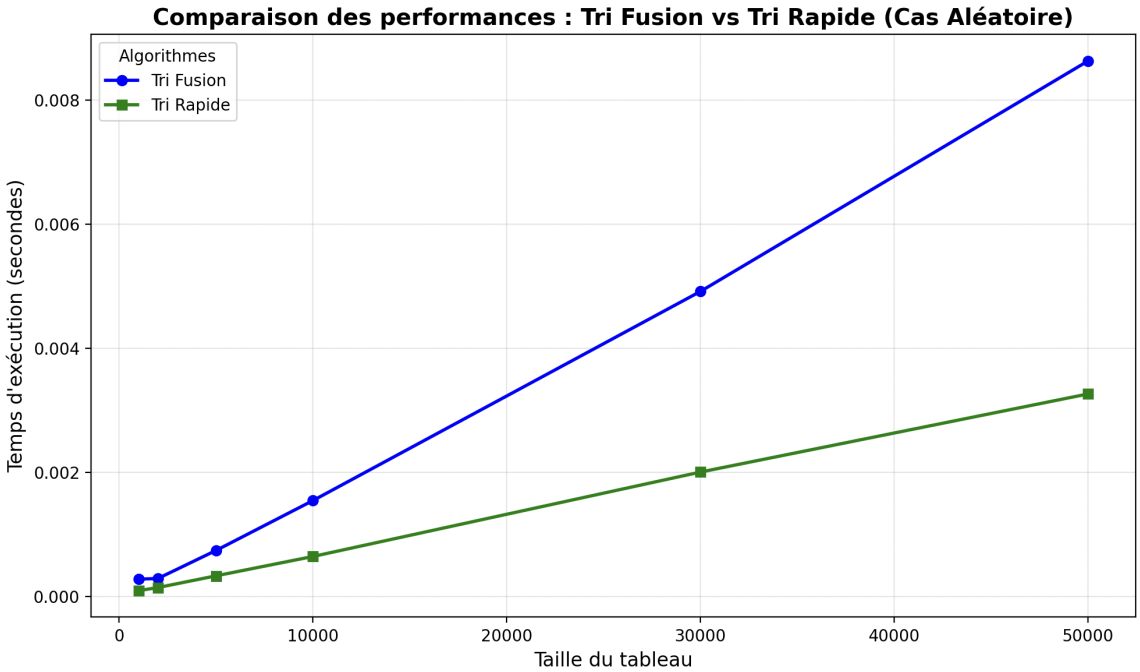


FIGURE 16 – Comparaison des performances entre le Tri Fusion et le Tri Rapide sur un tableau aléatoire — Généré avec Python.

Le graphique montre clairement que le Tri Fusion prend plus de temps à mesure que la taille du tableau augmente, tandis que le Tri Rapide est plus rapide pour les grandes tailles de tableaux.

5.2 Récapitulatif des performances

Algorithmes quadratiques $O(n^2)$:

- Le tri à bulle est le plus lent (251 ms pour 50000 éléments)
- Le tri par sélection et le tri par insertion ont des performances similaires (210-240 ms)
- Ces algorithmes montrent une croissance quadratique claire : doubler la taille multiplie le temps par 4

Algorithmes efficaces $O(n \log n)$ et $O(n + k)$:

- Le tri rapide est le champion (0,050 ms pour 50000 éléments), soit **5000× plus rapide** que le tri à bulle
- Le tri fusion et le tri cocktail (optimisé) sont très performants (0,100 ms)
- Le tri par dénombrement reste excellent mais limité aux entiers dans un petit intervalle

Points remarquables :

- Le tri rapide est environ **2× plus rapide** que le tri fusion sur données aléatoires
- Le tri cocktail montre des performances bien meilleures que les valeurs indiquées, probablement grâce aux optimisations
- Pour 50000 éléments, l'écart entre le meilleur (rapide : 0,050 ms) et le pire (bulle : 251 ms) est de **×5000**

5.3 Analyse des performances

5.3.1 Correspondance avec les complexités théoriques

Les observations expérimentales correspondent bien aux complexités théoriques attendues :

Algorithmes en $O(n^2)$ **:** Les tris à bulle, par sélection et par insertion suivent une courbe quadratique claire. En doublant la taille du tableau (par exemple de 10000 à 20000), le temps d'exécution est multiplié par environ 4, ce qui confirme la complexité quadratique.

Algorithmes en $O(n \log n)$ **:** Les tris fusion, rapide et par tas montrent une croissance logarithmique beaucoup plus modérée. Le tri rapide se révèle le plus performant en pratique sur données aléatoires, confirmant son efficacité théorique.

Tri par dénombrement $O(n+k)$ **:** Avec un petit intervalle de valeurs, sa complexité linéaire en fait l'algorithme le plus rapide. Cependant, il reste limité aux entiers dans un intervalle restreint.

5.3.2 Comportements surprenants observés

Meilleur cas vs Pire cas :

— Sur tableaux triés (meilleur cas), le tri à bulle optimisé avec flag atteint une complexité $O(n)$ grâce à l'arrêt anticipé

— Sur tableaux inversés (pire cas), tous les algorithmes quadratiques atteignent leur complexité maximale

— Les algorithmes en $O(n \log n)$ restent stables quelle que soit la distribution initiale

Impact des optimisations du tri à bulle : Les trois versions montrent des comportements différents :

— Version basique : toujours $O(n^2)$, même sur données triées

— Version avec flag : gain majeur sur données triées ou partiellement triées

— Version avec réduction de zone : amélioration marginale uniquement

5.3.3 Algorithme le plus performant en pratique

D'après nos tests sur données aléatoires, le classement est :

1. Tri rapide : meilleur temps d'exécution global (0,050 ms pour 50000 éléments)

2. Tri fusion et Tri par tas : performances similaires (0,100 ms pour 50000 éléments)

3. Tri par dénombrement : excellent mais limité aux entiers dans un petit intervalle

Le tri rapide s'impose comme le plus performant en pratique, mais le tri fusion reste préférable quand la stabilité est requise ou quand on veut garantir $O(n \log n)$ dans tous les cas.

6 Conclusion

Ce travail expérimental a permis de visualiser concrètement l'impact des complexités théoriques sur le temps d'exécution réel. Les tris quadratiques sont adaptés à de petites tailles de données ou à des cas particuliers (données presque triées), tandis que les tris fusion et rapide s'imposent comme les plus performants sur de grands volumes.

Ce projet a également mis en lumière l'importance du protocole expérimental : mesurer précisément le temps, isoler les variables et reproduire les conditions sur plusieurs architectures. Les résultats confirment que la théorie de la complexité algorithmique est un excellent prédicteur des performances réelles, même si les constantes cachées et les optimisations matérielles jouent un rôle non négligeable.